Modbus TCP MITM Attack on Industrial Control Systems

Sensor Spoofing via Man-in-the-Middle Proxy on OT Network

CVSS 9.1	CRITICAL	PROTOCOL	Modbus TCP
Score	Severity	Type	Target

Author	Vaibhav Sharma
Environment	Factory I/O + OpenPLC Runtime (Docker) + Kali Linux VM
Attack Type	Man-in-the-Middle | Sensor Spoofing | FC=2 Response Injection
Tools Used	Python 3, Scapy, Socket, VMware, Wireshark, OpenPLC Editor
Classification	Educational / Personal Research — Isolated Lab Only

■■ For educational purposes only. All testing performed in an isolated lab environment.

1. Executive Summary

This report documents a successful Man-in-the-Middle (MITM) attack against an Industrial Control System (ICS) built on the Modbus TCP protocol. The attacker, positioned between a Programmable Logic Controller (OpenPLC) and factory simulation software (Factory I/O), intercepted and modified real-time sensor data — causing the PLC to make incorrect control decisions based entirely on falsified inputs.

Modbus TCP, first standardized in 1979, powers the majority of global industrial automation infrastructure. Its design predates modern cybersecurity thinking and includes zero native protections: no authentication, no encryption, and no integrity verification. This makes it trivially exploitable from any network-adjacent position.

KEY FINDINGS

- Modbus TCP has NO authentication — any device can impersonate another
- FC=2 sensor responses can be silently modified mid-transit
- Attack requires only Python stdlib — no specialized tools needed
- PLC makes decisions on spoofed data with zero indication of compromise
- Detection is near-impossible without dedicated OT monitoring tools

2. Lab Architecture & Environment

Component	Tool / Version	Role in Lab	IP / Port
Factory I/O v2.5	Windows 10 Host	ICS Physical Simulation + Modbus Server	192.168.0.196:503
OpenPLC Runtime v4	Docker (Windows)	PLC Master — reads sensors, controls conveyor	r 172.x.x.x (Docker)
OpenPLC Editor	Windows	Ladder logic programming environment	localhost:8080
Kali Linux 2026.1	VMware Workstation	Attacker machine — MITM Proxy	192.168.0.185:503
Python 3 MITM Proxy	scripts.py	Intercepts + spoofs FC=2 Modbus responses	Port 503 (relay)
VMware Workstation	Host	Virtualization for isolated lab network	NAT / Wi-Fi bridge

Attack Flow Diagram:

```
OpenPLC (Docker) ■■[TCP:503]■■■ Kali MITM Proxy ■■[TCP:503]■■■ Factory I/O (Windows) ■ Intercepts FC=2 Response Modifies Byte[9]: 0x00 → 0xFF Forwards spoofed packet Result: PLC believes sensor = CLEAR → Conveyor stays ON → Box falls off end
```

3. Vulnerability Deep Dive

3.1 Modbus TCP Protocol Structure

Modbus TCP encapsulates the Modbus Application Data Unit (ADU) inside a TCP/IP frame with a 6-byte Modbus Application Protocol (MBAP) header. The complete frame structure is:

```
MBAP Header: Bytes [0-1] Transaction ID — Matches request to response Bytes [2-3] Protocol ID — Always 0x0000 for Modbus TCP Bytes [4-5] Length — Remaining bytes in frame Byte [6] Unit ID — Slave device identifier PDU (Protocol Data Unit): Byte [7] Function Code — FC=1 Coils, FC=2 Discrete, FC=3 Holding... Bytes [8-9] Address / Data — Depends on function code Bytes [10+] Data payload — Values, quantities, status
```

3.2 FC=2 (Read Discrete Inputs) Analysis

Function Code 2 is used by the PLC (master) to read digital sensor states from field devices. In this lab, OpenPLC sends FC=2 requests every ~15ms to read the beam sensor status from Factory I/O.

Frame Type	Byte[7]	Byte[8]	Byte[9]	Byte[10]	Byte[11]
FC=2 Request (OpenPLC→FactIO)	0x02	Addr High 0x00	Addr Low 0x00	Qty High 0x00	Qty Low 0x01
FC=2 Response (FactIO→OpenPLC)	0x02	Byte Count 0x01	Sensor 0x00=blocked 0xFF=clear	—	—
SPOOFED Response (Kali→OpenPLC)	0x02	**Byte Count 0x01**	FORCED 0xFF (fake)	—	—

3.3 Why Modbus Has No Security

Modbus was designed in 1979 by Modicon for serial (RS-232/485) communication in closed, air-gapped factory floors. Security was irrelevant — physical access was the security. When Modbus was adapted to TCP/IP in 1999 (Modbus/TCP specification), no security features were added. The assumption of physical isolation remained, despite the protocol now running on open IP networks.

Security Feature	TCP/IP Standard	Modbus TCP	Impact if Missing
Authentication	TLS Certificates	NONE	Anyone can send commands
Encryption	TLS 1.3	NONE	Traffic readable in plaintext
Integrity Check	TLS MAC/HMAC	NONE	Packets can be modified
Session Management	TLS Sessions	NONE	No session binding
Access Control	Firewall/ACL	NONE (protocol-level)	No per-command auth
Audit Logging	Syslog/SIEM	NONE (protocol-level)	Attacks leave no trace

4. Attack Methodology

01 Reconnaissance

Identified Modbus TCP traffic using network scanner. Confirmed OpenPLC connects to Factory I/O on port 503. Mapped IP addresses using ipconfig and ip addr show.

02 Network Positioning

Kali VM placed on same network segment (192.168.0.0/24). Verified connectivity with ping. Configured Windows Firewall to allow ports 502/503.

03 Traffic Redirection

Modified OpenPLC Editor Remote Device config to point to Kali VM IP (192.168.0.185) instead of Factory I/O directly. OpenPLC now connects through attacker.

04 Proxy Deployment

Launched Python MITM proxy on Kali (scripts.py). Proxy listens on port 503, accepts OpenPLC connections, and relays to actual Factory I/O on port 503.

05 Packet Analysis

Monitored live Modbus traffic. Identified FC=2 (Read Discrete Inputs) response packets. Located sensor value at byte index [9] of response frame.

06 Sensor Spoofing

Modified FC=2 response Byte[9] from 0x00 (blocked) to 0xFF (unbroken). Forwarded modified packet to OpenPLC. PLC received falsified sensor state.

07 Impact Verification

Observed conveyor belt continuing to operate even when physical box blocked sensor beam. Attack confirmed successful — safety logic bypassed.

4.1 Core Attack Code

```
def modify_response(data: bytes) -> bytes: if len(data) < 10: return data fc = data[7] # Function Code if fc == 2: # FC=2 = Read Discrete Inputs modified = bytearray(data) modified[9] = 0xFF # Byte[9] = Sensor value # 0x00=blocked, 0xFF=clear (SPOOFED) return bytes(modified) return data
```

5. Impact & Risk Analysis

5.1 CVSS v3.1 Score Breakdown

Metric	Value	Score Contribution
Attack Vector (AV)	Network	High impact — remote exploitable
Attack Complexity (AC)	Low	High impact — no special conditions
Privileges Required (PR)	None	High impact — no auth needed
User Interaction (UI)	None	High impact — fully automated
Scope (S)	Changed	Affects beyond vulnerable component
Confidentiality (C)	None	No data leakage
Integrity (I)	High	Sensor data fully falsifiable
Availability (A)	High	Can disrupt industrial process
CVSS Base Score	**9.1**	**CRITICAL**

5.2 Real-World Impact Scenarios

Safety System Bypass	**CRITICAL**
Safety interlocks rely on sensor data. Spoofed sensors can disable emergency stops, allowing machinery to operate in unsafe conditions. In real plants: worker injury risk.	
Process Sabotage	**HIGH**
Production line can be disrupted without physical access. Attacker can selectively bypass or trigger sensors to cause defective products or line stoppages.	
Equipment Damage	**HIGH**
Conveyors, robotic arms, and actuators can be driven beyond safe operating parameters when sensor feedback is falsified.	
Covert Persistence	**HIGH**
Attack leaves no trace in standard Modbus/PLC logs. Can persist for extended periods undetected without specialized OT monitoring.	
Data Integrity Loss	**CRITICAL**
All historical sensor data recorded during attack period is corrupted. Audit trails and production records are unreliable.	

6. Mitigation Recommendations

Immediate

• Block Modbus port 502/503 from any unauthorized IP using firewall rules
• Isolate OT network from IT network physically or via firewall DMZ
• Monitor for unexpected Modbus connections using netstat/Wireshark
Short Term
• Deploy ModbusTLS (RFC draft) for encrypted, authenticated Modbus communication
• Implement IP allowlisting at network level — only known PLC IPs can send Modbus
• Install OT-specific IDS: Claroty, Dragos, or Nozomi Networks
Long Term
• Adopt Purdue Model / IEC 62443 architecture for defense-in-depth
• Migrate to OPC-UA (has built-in security) where feasible
• Regular OT penetration testing and red team exercises
• Implement sensor value anomaly detection (ML-based or rule-based)

7. Skills Demonstrated

Domain	Skills	Tools
ICS/OT Security	PLC programming, Modbus protocol, sensor logic, OT attack	OpenPLC , Factory I/O
Network Security	MITM attacks, packet interception, TCP/IP layer analysis	Wireshark, tcpdump, Python sockets
Python Development	Socket programming, multithreading, binary data manipulation	Construct, threading, socket
Linux Penetration	Kali Linux, privilege escalation, network tools, raw sockets	Kali Linux, VMware, ip/netstat
Protocol Analysis	Modbus TCP frame parsing, FC code analysis	Wireshark, Python struct
Security Research	Vulnerability identification, CVE-style writeup, PoC develop	CVSS calculator, documentation